TP3 : normalisation, mots vides, racines et lemmes

Traitement Automatique du Langage Naturel · EMSI · Chapitre 2 (partie 3) · Séance 8

Objectifs

1. utiliser et **adapter** une liste de mots vides (sans perdre la négation) ;
2. comparer la **racinisation** (Snowball, NLTK) et la **lemmatisation** (spaCy), en français et en anglais ;
3. **normaliser** un texte arabe (darija) ;
4. **mesurer** la taille du vocabulaire après chaque étape et **justifier** les choix.

Mode d'emploi

- Complétez les cellules marquées `# À compléter` (remplacez les `...`).
- Exécutez ensuite la cellule ✅ **Test** qui suit : elle doit afficher ✅.
- Répondez aux questions ✍️ dans la cellule « Votre réponse » placée juste en dessous.

Livrable attendu

Le tableau comparatif commenté de la partie 4 et la fonction `normaliser_tp3()`.

Partie 0 : préparation

0.1 Identification

Complétez vos informations puis exécutez la cellule : elle affiche le **nom du fichier** sous lequel vous devez enregistrer et rendre votre notebook.

```
NOM = ""
PRENOM = ""
GROUPE = ""

assert NOM and PRENOM and GROUPE, "Complétez NOM, PRENOM et GROUPE"
nom_fichier = f"TP3_{NOM}_{PRENOM}_{GROUPE}.ipynb".replace(" ", "_")
print("Enregistrez votre notebook sous le nom :", nom_fichier)
```

0.2 Installation et imports

Sur **Google Colab**, décommentez les 3 premières lignes la première fois (NLTK est déjà installé). Sur votre machine : `pip install nltk spacy pandas matplotlib`, puis téléchargez les deux modèles spaCy.

```
# !pip install -q spacy
# !python -m spacy download fr_core_news_sm
# !python -m spacy download en_core_web_sm

import re, html, unicodedata
from collections import Counter

import nltk
nltk.download("stopwords", quiet=True)
from nltk.corpus import stopwords
from nltk.stem.snowball import SnowballStemmer
from nltk.stem import PorterStemmer
from nltk.stem.isri import ISRIStemmer

import spacy
import pandas as pd
import matplotlib.pyplot as plt

nlp_fr = spacy.load("fr_core_news_sm")    # modèle français
nlp_en = spacy.load("en_core_web_sm")     # modèle anglais
print("NLTK", nltk.__version__, "| spaCy", spacy.__version__, "| pandas", pd.__version__)
```

0.3 Le corpus fil rouge

Trois petits corpus d'avis sur des restaurants, chacun avec son étiquette (`"pos"` ou `"neg"`) :

- `AVIS` : 26 avis **en français** ;
- `DARIJA` : 8 avis en **darija** écrits en lettres arabes, et 2 en **arabizi** ;
- `ENGLISH` : 6 avis **en anglais**.

```
AVIS = [
    ("Le tajine était délicieux et le service très rapide. Je recommande !", "pos"),
    ("Service TRÈS lent, le plat est arrivé froid. Je ne recommande pas.", "neg"),
    ("Les serveurs sont aimables et le cadre est magnifique 😍", "pos"),
    ("Le couscous n'était pas bon du tout, trop salé.", "neg"),
    ("Trooop bien !!! Livraison rapide et repas encore chaud.", "pos"),
    ("Livraison en retard de 45 minutes, la commande était incomplète.", "neg"),
    ("Très bon rapport qualité-prix, les portions sont généreuses.", "pos"),
    ("Le restaurant est sale et le personnel n'est pas aimable.", "neg"),
    ("J'ai adoré le thé à la menthe et les pâtisseries marocaines.", "pos"),
    ("Les prix sont trop élevés pour des plats sans goût.", "neg"),
    ("Accueil chaleureux, on s'est régalés en famille. Merci @dar_tajine !", "pos"),
    ("Plus jamais ! Le poisson n'était pas frais.", "neg"),
    ("Cadre agréable, musique douce, serveurs souriants.", "pos"),
    ("Commande annulée sans explication, service client injoignable.", "neg"),
    ("Le meilleur restaurant de Rabat, les brochettes sont excellentes.", "pos"),
    ("Les frites étaient froides et la sauce n'avait aucun goût.", "neg"),
    ("tres bon resto, je reviendrai avec mes amis", "pos"),
    ("Pas bon, pas chaud, pas rapide : rien ne va.", "neg"),
    ("Réservation facile sur https://dar-tajine.ma et accueil parfait.", "pos"),
    ("Le service était lent mais les plats étaient bons.", "pos"),
    ("Je ne suis pas satisfait, le livreur était impoli.", "neg"),
    ("Une expérience inoubliable, tout était parfait !", "pos"),
    ("Le restaurent est magnifque, les serveurs très gentils.", "pos"),
    ("Les plats sont arrivés froids, je ne commanderai plus jamais.", "neg"),
    ("Le plat est bon.", "pos"),
    ("Le plat n'est pas bon.", "neg"),
]

DARIJA = [
    ("الطاجين كان بنين بزاف و السيرفيس مزيان", "pos"),
    ("الماكلة باردة و الثمن غالي، ما عجبنيش", "neg"),
    ("المطعـــــم زوين و الخدمة سريعة 👍", "pos"),
    ("ما غاديش نرجع ليه، الخدمة خايبة", "neg"),
    ("أحسن مطعم فالرباط، شُكْرًا بزاااف", "pos"),
    ("احسن طاجين كليتو، الأثمنة مناسبة", "pos"),
    ("التوصيل تعطل ساعة و الماكلة ما كانتش سخونة", "neg"),
    ("إلى بغيتي تاكل مزيان سير لهاد المطعم", "pos"),
    ("lmakla bnina bzaf, 3jbatni", "pos"),           # arabizi
    ("service 5ayb, ma 3jbnich", "neg"),             # arabizi
]

ENGLISH = [
    ("The waiters were friendly and the dishes tasted amazing.", "pos"),
    ("We waited two hours and the fries were cold.", "neg"),
    ("Best restaurant in Rabat, the tagines are delicious!", "pos"),
    ("The rooms were dirty and the staff wasn't helpful.", "neg"),
    ("The delivery arrived late and the boxes were broken.", "neg"),
    ("Lovely views, generously served meals and cheaper prices.", "pos"),
]
print(len(AVIS), "avis FR |", len(DARIJA), "avis darija |", len(ENGLISH), "avis EN")
print(Counter(etiquette for _, etiquette in AVIS))
```

0.4 Le prétraitement du TP2 et l'analyse spaCy

La fonction `preprocess()` ci-dessous est une **version simplifiée** de celle du TP2 (vous pouvez la remplacer par la vôtre). Elle remplace les liens par `<URL>`, les mentions par `<USER>` et réduit les allongements.

Pour que spaCy garde `<URL>` et `<USER>` en **un seul token**, on lui déclare deux **cas particuliers**.

```python
def preprocess(texte):
    """Prétraitement du TP2 (version simplifiée)."""
    texte = html.unescape(texte)                          # &amp;amp; -> &
    texte = re.sub(r"https?://\S+", " <URL> ", texte)  # liens
    texte = re.sub(r"@\w+", " <USER> ", texte)         # mentions
    texte = re.sub(r"(\w)\1{2,}", r"\1\1", texte)      # trooop -> troop
    return re.sub(r"\s+", " ", texte).strip()

for jeton in ["<URL>", "<USER>"]:                         # 1 token, pas 3
    nlp_fr.tokenizer.add_special_case(jeton, [{"ORTH": jeton}])

textes = [preprocess(t) for t, _ in AVIS]
docs = list(nlp_fr.pipe(textes))                          # analyse de tout le corpus
print(textes[18])
print([t.text for t in docs[18]])
```

Partie 1 : les mots vides

Exercice 1.1 : comparer les listes de NLTK et de spaCy

Les deux bibliothèques fournissent une liste de mots vides français. Comparez-les avec les **opérations sur les ensembles** : `&` (intersection), `-` (différence).

👉 La liste de spaCy se trouve dans `nlp_fr.Defaults.stop_words`.

```python
sw_nltk = set(stopwords.words("french"))
sw_spacy = ...          # À compléter : la liste de spaCy, convertie en set

communs = ...           # À compléter : mots présents dans les DEUX listes
seulement_nltk = ...    # À compléter : mots présents seulement dans NLTK
seulement_spacy = ...   # À compléter : mots présents seulement dans spaCy

print("NLTK :", len(sw_nltk), "| spaCy :", len(sw_spacy), "| communs :", len(communs))
print("Seulement NLTK (15 premiers) :", sorted(seulement_nltk)[:15])
print("Seulement spaCy (15 premiers) :", sorted(seulement_spacy)[:15])
```

```python
# ✅ Test 1.1 (ne pas modifier)
assert isinstance(sw_spacy, set) and "le" in sw_spacy, "sw_spacy doit être un set (utilisez
set(...))"
assert communs == sw_nltk & sw_spacy, "communs : mots présents dans les deux listes (opérateur
&)"
assert seulement_nltk == sw_nltk - sw_spacy, "seulement_nltk : opérateur -"
assert seulement_spacy == sw_spacy - sw_nltk, "seulement_spacy : opérateur -"
print("✅ Exercice 1.1 réussi")
```

Observez : NLTK contient `"l"`, `"n"`, `"qu"` **sans apostrophe**, alors que spaCy découpe `l'`, `n'`, `qu'` **avec** l'apostrophe. Vérifions quels mots de la négation sont dans chaque liste :

```python
for mot in ["ne", "n'", "n", "pas", "plus", "jamais", "rien", "aucun", "sans", "très", "trop",
"mais"]:
    print(f"{mot:7} NLTK : {str(mot in sw_nltk):5}  spaCy : {mot in sw_spacy}")
```

✍️ **Question 1.1** : (a) Quelle liste est la plus longue ? (b) Pourquoi la liste de NLTK ne retire-t-elle pas le token `n'` produit par spaCy ? (c) Quels mots de la négation sont dans les deux listes ?

Votre réponse :

Exercice 1.2 : quelle part du corpus représentent les mots vides ?

1. Construisez la liste `mots` de tous les tokens **alphabétiques** du corpus, en minuscules (attributs spaCy `t.is_alpha` et `t.lower_`).
2. Comptez-les avec `Counter` et affichez les 10 plus fréquents.
3. Calculez la **proportion** de tokens qui sont des mots vides (liste spaCy).

```
mots = ...        # À compléter : tokens alphabétiques du corpus, en minuscules
freq = ...        # À compléter : Counter des mots
top10 = ...       # À compléter : les 10 mots les plus fréquents

nb_vides = ...    # À compléter : nombre de tokens qui sont des mots vides (liste spaCy)
proportion = ...  # À compléter : nb_vides divisé par le nombre total de tokens

print(len(mots), "tokens,", len(freq), "mots différents")
print(top10)
print(f"Mots vides : {nb_vides} tokens, soit {proportion:.0%} du corpus")
```

```
# ✅ Test 1.2
assert isinstance(mots, list) and all(m == m.lower() and m.isalpha() for m in mots), "mots :
tokens alphabétiques en minuscules"
assert freq == Counter(mots), "freq doit être Counter(mots)"
assert len(top10) == 10 and top10[0] == freq.most_common(1)[0], "top10 : freq.most_common(10)"
assert abs(proportion - sum(m in sw_spacy for m in mots) / len(mots)) < 1e-9, "proportion
incorrecte"
print("✅ Exercice 1.2 réussi")
```

Exercice 1.3 : retirer les mots vides… et perdre la négation

Écrivez la fonction `retirer_mots_vides(tokens, mots_vides)` qui renvoie la liste des tokens qui ne sont **ni** des mots vides **ni** de la ponctuation. `tokens` est une liste de tokens spaCy ; on renvoie leur forme en minuscules (`t.lower_`).

```
def retirer_mots_vides(tokens, mots_vides):
    # À compléter : renvoyer t.lower_ pour chaque token t qui n'est
    # ni de la ponctuation, ni un mot vide
    ...

for phrase in ["Le plat est bon.", "Le plat n'est pas bon.", "Le plat n'est jamais bon."]:
    doc = nlp_fr(phrase)
    print(f"{phrase:28}", retirer_mots_vides(doc, sw_spacy))
```

```
# ✅ Test 1.3
assert retirer_mots_vides(nlp_fr("Le service est rapide !"), sw_spacy) == ["service",
"rapide"], "retirez les mots vides ET la ponctuation, en minuscules"
assert retirer_mots_vides(nlp_fr("Le plat n'est pas bon."), set()) == ["le", "plat", "n'",
"est", "pas", "bon"], "avec une liste vide, seule la ponctuation doit partir"
print("✅ Exercice 1.3 réussi")
```

✍️ **Question 1.3** : Comparez les deux premières lignes. Quel est le problème pour l'**analyse de sentiment** ? Pourquoi « jamais » est-il conservé ?

Votre réponse :

Exercice 1.4 : construire une liste adaptée au corpus

On part de la liste de spaCy et :

1. on **retire** de la liste les mots de la négation (`NEGATION`) et d'intensité (`INTENSITE`), qu'on veut **garder** ;
2. on **ajoute** des mots du domaine, très fréquents dans tous les avis et donc peu utiles pour les distinguer.

Pour choisir les mots du domaine, la cellule suivante calcule la **fréquence documentaire** : le nombre d'avis dans lesquels chaque mot (non vide) apparaît.

```
df = Counter()
for doc in docs:
    df.update({t.lower_ for t in doc if t.is_alpha and t.lower_ not in sw_spacy})
print(df.most_common(10))
```

```
NEGATION = {"ne", "n'", "pas", "plus", "jamais", "rien", "aucun", "aucune", "sans", "ni"}
INTENSITE = {"très", "trop", "peu"}
MOTS_DOMAINE = set()    # À compléter : vos mots du domaine (choisis grâce à df)

mots_vides_tp = ...     # À compléter : liste spaCy, SANS négation ni intensité, PLUS les mots du domaine

print(len(mots_vides_tp), "mots vides dans la liste adaptée")
for phrase in ["Le plat est bon.", "Le plat n'est pas bon.", "Le restaurant est très sale."]:
    print(f"{phrase:30}", retirer_mots_vides(nlp_fr(phrase), mots_vides_tp))
```

```
# ✅ Test 1.4
assert isinstance(mots_vides_tp, set), "mots_vides_tp doit être un set"
assert "le" in mots_vides_tp and "est" in mots_vides_tp, "les mots vides habituels doivent rester"
assert not ({"pas", "ne", "n'", "jamais", "très"} & mots_vides_tp), "la négation et l'intensité doivent être GARDÉES"
assert MOTS_DOMAINE and MOTS_DOMAINE <= mots_vides_tp, "ajoutez vos mots du domaine"
assert retirer_mots_vides(nlp_fr("Le plat est bon."), mots_vides_tp) != retirer_mots_vides(nlp_fr("Le plat n'est pas bon."), mots_vides_tp)
print("✅ Exercice 1.4 réussi : la négation est conservée")
```

✍️ **Question 1.4** : Quels mots du domaine avez-vous ajoutés ? Justifiez avec `df`. Pourquoi ne faut-il pas ajouter « service » ou « bon » ?

Votre réponse :

Partie 2 : racines (Snowball) et lemmes (spaCy)

2.1 À la main d'abord

Avant d'exécuter le code, **prédisez** la racine que Snowball va donner pour chaque mot, puis vérifiez avec l'exercice 2.1. Rappel : Snowball **coupe les suffixes** (-ons, -ait, -ement, -eur…) sans dictionnaire.

Mot	Votre prédiction	Snowball
mangeons		
rapidement		
livraison		
livreur		
serveuse		

Exercice 2.1 : la racinisation avec Snowball

Complétez la fonction `raciner(mots)` qui renvoie la liste des racines, puis la fonction `nb_racines(mots)` qui renvoie le **nombre de racines différentes** d'une famille de mots.

```python
stemmer_fr = SnowballStemmer("french")

def racine(mots):
    # À compléter : renvoyer la liste des racines des mots
    ...

def nb_racines(mots):
    # À compléter : renvoyer le nombre de racines DIFFÉRENTES
    ...

familles = {
    "manger":  ["mange", "mangeons", "mangeait", "mangé", "mangeur"],
    "rapide":  ["rapide", "rapides", "rapidement", "rapidité"],
    "livrer":  ["livrer", "livraison", "livreur", "livré"],
    "servir":  ["servir", "service", "serveur", "serveuse", "servi"],
    "aller":   ["aller", "allons", "va", "irai"],
}
for nom, mots_famille in familles.items():
    print(f"{nom:8} {nb_racines(mots_famille)} racine(s) :", racine(mots_famille))
```

```python
# ✅ Test 2.1
assert racine(["mangeait", "rapides"]) == ["mang", "rapid"], "racine doit renvoyer la LISTE des racines"
assert nb_racines(["mange", "mangé", "mangeait"]) == 1 and nb_racines(["va", "irai"]) == 2, "nb_racines : len(set(...))"
print("✅ Exercice 2.1 réussi")
```

Exercice 2.2 : la lemmatisation avec spaCy

Écrivez `lemmatiser(texte)` qui renvoie la liste des **lemmes en minuscules** des tokens d'un texte, sans la ponctuation. Un lemme s'obtient avec `t.lemma_`.

```python
def lemmatiser(texte):
    doc = nlp_fr(texte)
    # À compléter : renvoyer les lemmes en minuscules, sans la ponctuation
    ...

print(lemmatiser("Les serveurs étaient aimables et nous avons mangé des tajines délicieux."))
print(lemmatiser("Nous irons, tu vas, ils allaient."))
```

```python
# ✅ Test 2.2
assert lemmatiser("Les serveurs étaient aimables.") == ["le", "serveur", "être", "aimable"], "lemmes en minuscules, sans la ponctuation"
print("✅ Exercice 2.2 réussi")
```

Exercice 2.3 : tableau comparatif mot / racine / lemme

Pour chaque token (hors ponctuation) de l'avis ci-dessous, ajoutez une ligne `{"mot": ..., "racine": ..., "lemme": ..., "catégorie": ...}` à la liste `lignes`, puis affichez le tableau avec pandas. La catégorie s'obtient avec `t.pos_`.

```python
avis = "Les serveurs étaient souriants mais les livraisons arrivaient froides."
lignes = []
for t in nlp_fr(avis):
    # À compléter : si t n'est pas de la ponctuation, ajouter à lignes le dictionnaire
    # {"mot": ..., "racine": ..., "lemme": ..., "catégorie": ...}
    ...

tableau = pd.DataFrame(lignes)
tableau
```

```
# ✅ Test 2.3
assert list(tableau.columns) == ["mot", "racine", "lemme", "catégorie"], "4 colonnes : mot,
racine, lemme, catégorie"
assert len(tableau) == 9 and tableau.loc[0, "lemme"] == "le", "une ligne par token (9 tokens
hors ponctuation)"
print("✅ Exercice 2.3 réussi")
```

✍️ **Question 2.3** : Donnez un mot pour lesquel la racine n'est **pas un vrai mot** et un mot où le lemme est **plus utile** que la racine. Quel est l'avantage de la racine ?

Votre réponse :

Exercice 2.4 : et en anglais ?

Comparez **Porter**, **Snowball (english)** et les lemmes de **spaCy (anglais)** sur les tokens alphabétiques des avis `ENGLISH`. Construisez un tableau avec une ligne par mot **différent**.

```
porter = PorterStemmer()
snow_en = SnowballStemmer("english")

lignes_en = []
deja_vus = set()
for texte, _ in ENGLISH:
    for t in nlp_en(texte):
        # À compléter : pour chaque token ALPHABÉTIQUE pas encore vu (en minuscules),
        # l'ajouter à deja_vus et ajouter à lignes_en le dictionnaire
        # {"mot": ..., "Porter": ..., "Snowball": ..., "lemme spaCy": ...}
        ...

tableau_en = pd.DataFrame(lignes_en)
# On n'affiche que les mots où au moins une méthode change le mot
tableau_en[(tableau_en["Porter"] != tableau_en["mot"]) | (tableau_en["lemme spaCy"] !=
tableau_en["mot"])]
```

```
# ✅ Test 2.4
assert list(tableau_en.columns) == ["mot", "Porter", "Snowball", "lemme spaCy"]
ligne = tableau_en[tableau_en["mot"] == "were"].iloc[0]
assert ligne["lemme spaCy"] == "be" and ligne["Porter"] == "were"
print("✅ Exercice 2.4 réussi")
```

✍️ **Question 2.4** : Repérez dans le tableau (a) un verbe irrégulier que seul spaCy ramène à sa forme de base, (b) un mot que Porter coupe de façon illisible, (c) une différence entre Porter et Snowball.

Votre réponse :

Exercice 2.5 : repérer la sur-racinisation dans le corpus

Une racine peut regrouper des mots de **sens différents**. Pour le vérifier, on associe à chaque racine l'ensemble des **lemmes** qui la produisent. Une racine avec plusieurs lemmes est **suspecte**.

```
racine_vers_lemmes = {}
for doc in docs:
    for t in doc:
        # À compléter : pour chaque token alphabétique qui n'est pas dans mots_vides_tp,
        # calculer sa racine r et ajouter son lemme (minuscules) à l'ensemble
racine_vers_lemmes[r]
        ...

suspects = ...   # À compléter : les racines associées à au moins 2 lemmes différents
for r, lemmes in sorted(suspects.items()):
    print(f"{r:10} <- {sorted(lemmes)}")
```

```
# ✅ Test 2.5
assert all(len(l) >= 2 for l in suspects.values()) and len(suspects) >= 1
print("✅ Exercice 2.5 réussi :", len(suspects), "racine(s) suspecte(s)")
```

✍️ **Question 2.5** : Pour chaque racine affichée, s'agit-il d'une **vraie** sur-racinisation (sens différents) ou d'une **erreur du lemmatiseur** (le petit modèle spaCy se trompe) ? Que concluez-vous ?

Votre réponse :

Partie 3 : normaliser l'arabe (darija)

Exercice 3.1 : compléter `normaliser_arabe()`

Les étapes 1 et 2 sont données. Complétez les étapes 3 à 6 (voir le cours, partie 3) :

Étape	Opération	Exemple
3	formes de alef ا → آ إ أ	أحمد devient احمد
4	alef maqsura ي → ى	إلى devient الي
5	ta marbuta ه → ة	مدرسة devient مدرسه
6	chiffres arabes-indiens ٩…٠ → 0…9	٢٠٢٤ devient 2024

Les lettres sont écrites avec leur **code Unicode** (`"\u0623"` = أ) pour que le code reste lisible.

```
def normaliser_arabe(t):
    t = re.sub("[\u064B-\u0652]", "", t)     # 1. voyelles brèves (tachkil)
    t = t.replace("\u0640", "")              # 2. tatweel (allongement)
    # 3. À compléter : alef avec hamza (\u0623, \u0625, \u0622) -> alef (\u0627)
    # 4. À compléter : alef maqsura (\u0649) -> ya (\u064A)
    # 5. À compléter : ta marbuta (\u0629) -> ha (\u0647)
    # 6. À compléter : chiffres "٠١٢٣٤٥٦٧٨٩" -> "0123456789"
    return t

for mot in ["٢٠٢٤" ,"المطعـــــم" ,"شُكْرًا" ,"مدرسة" ,"إلى" ,"أحمد"]:
    print("avant :", mot, "| après :", normaliser_arabe(mot))
```

```
# ✅ Test 3.1
tests = {":"المطعـــــم" ,"شكرا" :"شُكْرًا" ,"مدرسه" :"مدرسة" ,"الي" :"إلى" ,"احمد" :"أحمد"
2024" :"٢٠٢٤" ,"المطعم""}
for avant, attendu in tests.items():
    assert normaliser_arabe(avant) == attendu, f"{avant} devrait donner {attendu}, pas
{normaliser_arabe(avant)}"
print("✅ Exercice 3.1 réussi")
```

Partie 4 : mesurer la taille du vocabulaire (livrable)

On applique les étapes **une par une** au corpus français et on mesure, après chaque étape :

- le **nombre de tokens** (la longueur totale des textes) ;
- la **taille du vocabulaire** (le nombre de mots **différents**).

La fonction `sans_accents()` du cours est fournie.

```
def sans_accents(mot):
    nfd = unicodedata.normalize("NFD", mot)
    return "".join(c for c in nfd if unicodedata.category(c) != "Mn")

print(sans_accents("Très élevés, ça coûte cher"))
```

Exercice 4.1 : deux fonctions de mesure

`listes` est une liste de listes de tokens (une liste par avis).

```
def nb_tokens(listes):
    # À compléter : nombre total de tokens dans toutes les listes
    ...

def taille_vocabulaire(listes):
    # À compléter : nombre de tokens DIFFÉRENTS dans toutes les listes
    ...

exemple = [["le", "plat", "est", "bon"], ["le", "plat", "est", "froid"]]
print(nb_tokens(exemple), taille_vocabulaire(exemple))
```

```
# ✅ Test 4.1
assert nb_tokens(exemple) == 8 and taille_vocabulaire(exemple) == 5, "attendu : 8 tokens et 5 mots différents"
assert nb_tokens([]) == 0 and taille_vocabulaire([[]]) == 0, "cas des listes vides"
print("✅ Exercice 4.1 réussi")
```

Exercice 4.2 : le tableau comparatif

Complétez le dictionnaire `etapes` : chaque étape associe à chaque avis la liste de ses tokens transformés. Les étapes sont **cumulatives** (chacune reprend la précédente). La fonction `garde(t)` (fournie) indique si un token est conservé après le retrait de la ponctuation et des mots vides de **votre** liste adaptée.

```
def garde(t):
    return not t.is_punct and not t.is_space and t.lower_ not in mots_vides_tp

etapes = {
    "0. tokens bruts":        [[t.text for t in d] for d in docs],
    "1. minuscules":          [[t.lower_ for t in d] for d in docs],
    "2. sans ponctuation":    ...,   # À compléter
    "3. sans accents":        ...,   # À compléter (utilisez sans_accents)
    "4. sans mots vides":     ...,   # À compléter (utilisez garde)
    "5a. racines (Snowball)": ...,   # À compléter
    "5b. lemmes (spaCy)":     ...,   # À compléter
}

v0 = taille_vocabulaire(etapes["0. tokens bruts"])
resultats = pd.DataFrame([
    {"étape": nom, "tokens": nb_tokens(l), "vocabulaire": taille_vocabulaire(l),
     "réduction du vocabulaire": f"{1 - taille_vocabulaire(l) / v0:.0%}"}
    for nom, l in etapes.items()])
resultats
```

```
# ✅ Test 4.2
assert all(isinstance(v, list) and len(v) == len(docs) for v in etapes.values()), "une liste par avis pour chaque étape"
voc = list(resultats["vocabulaire"])
assert voc[0] >= voc[1] >= voc[2] >= voc[3] >= voc[4], "les étapes 0 à 4 doivent réduire le vocabulaire"
assert "pas" in {m for l in etapes["5b. lemmes (spaCy)"] for m in l}, "la négation doit être conservée"
print("✅ Exercice 4.2 réussi")
```

Exercice 4.3 : visualiser

Tracez un diagramme en barres horizontales de la taille du vocabulaire à chaque étape (`plt.barh`).

```
plt.figure(figsize=(8, 3.5))
# À compléter : barres horizontales (plt.barh) de resultats["vocabulaire"] en fonction de resultats["étape"]
# À compléter : première étape en haut, titre de l'axe x et titre du graphique
plt.tight_layout()
plt.show()
```

Exercice 4.4 : la fonction finale `normaliser_tp3()`

Assemblez **votre** chaîne de traitement dans une fonction réutilisable (TP6, TP8) : `preprocess` → spaCy → retrait de la ponctuation et des mots vides adaptés → **lemmes** (par défaut) ou **racines**.

```
def normaliser_tp3(texte, methode="lemmes"):
    doc = nlp_fr(preprocess(texte))
    if methode == "lemmes":
        return ...    # À compléter : lemmes en minuscules des tokens gardés
    elif methode == "racines":
        return ...    # À compléter : racines Snowball des tokens gardés
    raise ValueError("methode doit être 'lemmes' ou 'racines'")

for texte, etiquette in AVIS[:3] + AVIS[-2:]:
    print(etiquette, normaliser_tp3(texte))
```

```
# ✅ Test 4.4
assert normaliser_tp3("Les serveurs étaient aimables !") == ["serveur", "aimable"], "méthode lemmes : garde(t) et t.lemma_.lower()"
assert normaliser_tp3("Les serveurs étaient aimables !", "racines") == ["serveur", "aimabl"], "méthode racines : stemmer_fr.stem(t.lower_)"
assert normaliser_tp3("Le plat est bon.") != normaliser_tp3("Le plat n'est pas bon."), "la négation doit être conservée"
print("✅ Exercice 4.4 réussi : normaliser_tp3() est prête pour les TP suivants")
```

✍️ Conclusion (livrable)

Rédigez un **commentaire de 6 à 10 lignes** du tableau de l'exercice 4.2 en répondant à ces questions :

1. Quelle étape réduit le **plus** le vocabulaire ? Et le nombre de tokens ?
2. Racines ou lemmes : lequel réduit le plus le vocabulaire ? Pourquoi ?
3. Quelles informations a-t-on **perdues** en route (citez des exemples du corpus) ?
4. Pour l'**analyse de sentiment** du TP8, quelle chaîne de traitement choisissez-vous ? Justifiez.

Votre commentaire (6 à 10 lignes) :